

CS 2089M 数据库系统

2024–2025 春夏学期期末回忆卷

整理自 CC98 论坛*

共 9 大题：SQL 查询、E-R 模型、关系形式化、存储与文件结构、索引、
查询处理、查询优化、事务与并发控制、恢复

Problem 1: SQL Querying

```
CREATE TABLE train (  
    trnid INTEGER PRIMARY KEY,  
    name   VARCHAR(50),  
    type   INTEGER  
);
```

```
CREATE TABLE station (  
    staid      INTEGER PRIMARY KEY,  
    name       VARCHAR(100),  
    city       VARCHAR(50),  
    stop_sequence INTEGER,  
    rank       INTEGER  
);
```

```
CREATE TABLE timetable (  
    id          INTEGER PRIMARY KEY,  
    train       INTEGER,  
    station     INTEGER,  
    arrival_time TIME,  
    departure_time TIME,  
    FOREIGN KEY (train) REFERENCES train(trnid),  
    FOREIGN KEY (station) REFERENCES station(staid)  
);
```

```
CREATE TABLE ticket (  

```

*原卷说明：部分题目回忆细节有误，缺失数据已在相应位置标注。

```

tid    INTEGER PRIMARY KEY,
train  INTEGER,
src    INTEGER,
des    INTEGER,
price  DECIMAL(10,2),
date   DATE,
FOREIGN KEY (train) REFERENCES train(trnid),
FOREIGN KEY (src) REFERENCES station(staid),
FOREIGN KEY (des) REFERENCES station(staid)
);

```

1. **True or False:** Which of the following SQL queries correctly finds the name with the most stops in the `station` table?

主要考察查询最大值的多种方法。

Query A:

```

SELECT s.name, COUNT(t.station) AS stop_count
FROM station s
JOIN timetable t ON s.staid = t.station
ORDER BY stop_count DESC
LIMIT 1;

```

解: Query A 没有 `GROUP BY`，直接 `COUNT(t.station)` 会把全表聚合为一行，无法按站分组计数。掉一个 `GROUP BY`，逻辑错。

答案: × (False)

Query B:

```

SELECT s.name, COUNT(t.station) AS stop_count
FROM station s
JOIN timetable t ON s.staid = t.station
GROUP BY s.staid, s.name
ORDER BY stop_count DESC
LIMIT 1;

```

解: 正确。 `GROUP BY s.staid, s.name` 分组， `ORDER BY + LIMIT 1` 取最大值。语法和逻辑均无误。

答案: ✓ (True)

Query C:

```

SELECT s.name, COUNT(t.station) AS stop_count
FROM station s
JOIN timetable t ON s.staid = t.station
GROUP BY s.staid, s.name
HAVING COUNT(t.station) = (
    SELECT MAX(cnt)
    FROM (
        SELECT COUNT(station) AS cnt
        FROM timetable
        GROUP BY station
    ) counts
);

```

解：正确。子查询算出最大停靠次数，HAVING 筛出等于最大值的站。逻辑完美，且能处理并列第一。

答案：√ (True)

Query D:

```

SELECT s.name, COUNT(t.station) AS stop_count
FROM station s
JOIN timetable t ON s.staid = t.station
GROUP BY s.staid, s.name
HAVING COUNT(t.station) >= ALL (
    SELECT COUNT(station) AS cnt
    FROM timetable
    GROUP BY station
);

```

解：正确。>= ALL(subquery) 等价于 >= max(subquery)，筛出计数不小于所有其他计数的站。逻辑正确。

答案：√ (True)

2. **True or False:** Which of the following SQL queries correctly finds the cheapest price among tickets with origin A and destination B?

Query A:

```

SELECT t.tid
FROM ticket t
JOIN station src ON t.src = src.staid

```

```
JOIN station des ON t.des = des.staid
WHERE src.name = 'A' AND des.name = 'B'
ORDER BY t.price ASC
LIMIT 1;
```

解：正确。标准 JOIN ... ON 语法，两表联表找始发/终到站名，ORDER BY + LIMIT 1 取最低价票。

答案：✓ (True)

Query B:

```
SELECT t.tid
FROM ticket t, station src, station des
WHERE t.src = src.staid AND t.des = des.staid
      AND src.name = 'A' AND des.name = 'B'
ORDER BY t.price ASC
LIMIT 1;
```

解：正确。隐式 CROSS JOIN + WHERE 等价于显式 JOIN ... ON。结果与 Query A 完全相同。

答案：✓ (True)

Query C:

```
SELECT C.tid
FROM (
    SELECT t.tid, t.price
    FROM ticket t, station src, station des
    WHERE t.src = src.staid AND t.des = des.staid
          AND src.name = 'A' AND des.name = 'B'
) C
ORDER BY C.price ASC
LIMIT 1;
```

解：正确。子查询 C 先筛出符合条件的票，外层按价格排序取最小。等价于前两者，只是嵌套层次多了一层。

答案：✓ (True)

Query D:

```
SELECT t.tid
FROM ticket t
```

```

WHERE EXISTS (
    SELECT 1 FROM station src
    WHERE t.src = src.staid AND src.name = 'A'
)
AND EXISTS (
    SELECT 1 FROM station des
    WHERE t.des = des.staid AND des.name = 'B'
)
ORDER BY t.price ASC
LIMIT 1;

```

解：正确。用 EXISTS 半连接分别检查 A 始发和 B 到达，价格排序取最低。逻辑等价于 join 写法。

答案：✓ (True)

答案：四者全部正确。

3. **Fill in the blank** to implement the query: Find the name of trains that depart later than 17:00:00.

```

SELECT ____ FROM train
WHERE EXISTS (
    SELECT 1
    FROM timetable
    WHERE _____ AND
        departure_time > '17:00:00'
);

```

解：第一空：需要选的是 train 表的名字，故填 name (或 trnid, name)。

第二空:EXISTS 子查询需要把 train 表和 timetable 表关联起来:timetable.train = train.trnid (或 trnid)。

答案：第一空: name; 第二空: timetable.train = train.trnid。

4. **True or False:** When joining the timetable and station tables, the condition city = "Hangzhou" can be applied to station first before joining with timetable.

解：正确。下推选择(selection pushdown)是查询优化的基本规则:先在 station 上做 $\sigma_{\text{city}=\text{"Hangzhou"}}$, 缩减中间结果, 再与 timetable 连接。只要该条件仅涉及 station 的属性, 提前过滤不会改变结果 (等价于代数规则 $\sigma_p(r \bowtie s) = \sigma_p(r) \bowtie s$ 当 p 仅含 r 的属性)。

答案：✓ (True)

5. **True or False:** When joining the `timetable`, `station`, and `train` tables, the conditions `city = "Hangzhou"` and `train.name = "ABC"` can be applied to `station` and `train` first before joining with `timetable`.

解：正确。同理，两个选择条件分别仅涉及 `station` 和 `train` 的属性，都应尽早下推到对应表上，缩减参与 join 的行数。这是查询优化的基本操作。

答案：✓ (True)

6. **Write the relational algebra expression** for the following query: Find the name and type of trains that stop at the "Hangzhou Dong" station.

答案：

$$\pi_{\text{name,type}} \left(\sigma_{\text{station.name} = \text{'Hangzhou Dong'}} (\text{train} \bowtie_{\text{trnid} = \text{train}} \text{timetable} \bowtie_{\text{station} = \text{staid}} \text{station}) \right)$$

或等价地先将选择下推到 `station` 上再连接：

$$\pi_{\text{name,type}} \left(\text{train} \bowtie (\text{timetable} \bowtie \sigma_{\text{name} = \text{'Hangzhou Dong'}} (\text{station})) \right)$$

Problem 2: E-R Model

[原卷回忆细节有误]

Assumptions:

- A passenger can purchase multiple tickets.
 - A ticket may remain unsold.
 - A station must serve as the source (`src`) for at least one ticket.
 - `timetable` is a weak entity set, with `train` as its identifying entity set.
1. If `timetable` is a weak entity set, with `train` as its identifying entity set, how is the relationship set connecting them represented?

- A. undirected line
- B. undirected double line
- C. directed line
- D. directed double line

解：弱实体集与其标识实体集之间的**标识联系** (identifying relationship) 用**双线菱形**表示，连线到弱实体集一侧为**双线** (边框)，到强实体集一侧为普通线，标识联系本身用双线菱形表示。经典 E-R 记号：identifying relationship → 双线菱形；弱实体集边框为双线。

答案：D. directed double line (标识联系均双线)

- 2-5 What is the shape of edge *a*, *b*, *c*, *d*?

[原卷此处数据缺失] 原卷附有 E-R 图，图中标有边 *a*, *b*, *c*, *d*，图已丢失。选项均为：A. undirected line B. undirected double line C. directed line D. directed double line

答案：标注：此类题考查 E-R 图中各边的类型——普通边（单线无箭头）、标识联系边（双线）、箭头边（最多一）、基数约束边的辨识。需根据图中实体和联系的语义选择正确记号。具体答案依赖于缺失的图。

Problem 3: Relational Formalization

[原卷此处数据缺失] 原卷关系 R 及函数依赖集 F 的具体内容缺失。

1. **True or False:** The closures of several attribute sets contain the same attributes.

解：属性闭包的计算：给定 FD 集 F ，计算 $\{\alpha\}_F^+$ 。若两组属性集的闭包相同，说明它们互相确定（等价于 $\alpha \leftrightarrow \beta$ ）。

答案：需根据具体 FD 计算闭包后比较。

2. The canonical cover of F is: _____

解：正则覆盖计算步骤：

- (a) 将所有 FD 右部拆为单属性
- (b) 去除各 FD 左部多余属性（逐个尝试移除，检查闭包是否仍包含右部）
- (c) 去除冗余 FD（逐个尝试删除，检查删除后闭包是否仍包含原右部）

答案：需根据具体 FD 计算。

3. Which of the following is a candidate key of R ?

A–D: (选项值缺失) E. None of the above

解：候选键判定：找到一组属性集 α 满足 $\alpha_F^+ = R$ （能推出所有属性）且 α 的任何真子集都不满足（最小性）。

答案：需根据 R 和 F 逐一计算。

4. Adding which FD will NOT change the candidate key(s)?

A–D: (选项值缺失) E. None of the above

解：向 F 添加一个 FD 后，若原候选键的闭包仍为 R 且不引入新的更小候选键，则候选键不变。若新 FD 的右部已是原键闭包的一部分，则不影响。

答案：需根据具体候选键和选项判断。

5. **True or False:** Decomposing R into $R_1(\dots)$ and $R_2(\dots)$ is NOT lossless. [原卷此处数据缺失]
分解属性缺失。

解：无损连接分解判定：设分解为 R_1 和 R_2 ，则需 $R_1 \cap R_2 \rightarrow R_1$ 或 $R_1 \cap R_2 \rightarrow R_2$ 在 F^+ 中成立。即公共属性构成其中一个子模式的超键。

答案：需计算 $(R_1 \cap R_2)_F^+$ ，检查是否包含 R_1 或 R_2 。

6. **True or False:** Decomposing R into R_1, R_2, R_3 results in all three subrelations satisfying BCNF.

[原卷此处数据缺失] 分解属性缺失。

解：BCNF 判定：对每个子模式 R_i ，其上的每个非平凡 FD $\alpha \rightarrow \beta$ 都需满足 α 是 R_i 的超键。从 F 仅保留涉及 R_i 属性的 FD（或求投影 $F^+|_{R_i}$ ）后逐一检查。

答案：需根据各子模式上的局部 FD 验证是否为超键。

Problem 4: Storage and File Structure

[原卷回忆细节有误]

1. True or False:

- (a) Cache is primary storage and main memory is secondary storage.
- (b) DBMS transfers data from disk to main memory.
- (c) Data on the inner tracks of a disk is accessed slower than data on the outer tracks.
- (d) A 15,000 RPM disk does not necessarily have a shorter seek time than a 7,500 RPM disk.

解：

- (a) **False**。Cache 是 SRAM（更靠近 CPU），Main Memory 是 DRAM（主存）。两者都属于主存储（primary storage）。Disk/SSD 是二级存储（secondary storage）。故说 cache 是 primary 而 main memory 是 secondary 明显错误。
- (b) **True**。DBMS 从磁盘读数据时，先将磁盘块读入内存缓冲区（buffer pool），在内存中处理。这是 DBMS 存储管理的基本操作。
- (c) **True**。现代磁盘外道周长更大、线速度更高，且外道扇区密度通常更高，因此外道数据传输速率高于内道。
- (d) **True**。Seek time 主要由磁头臂的机械移动决定，与 RPM 关系不大。高 RPM 主要缩短的是旋转延迟（rotational latency），不保证 seek time 更短。

答案： (a) ×, (b) ✓, (c) ✓, (d) ✓

2. Given a record (23, "Chicago", 10), fill in its memory structure.

```
CREATE TABLE A (  
    id    INTEGER PRIMARY KEY,  
    name  VARCHAR,  
    type  INTEGER  
)
```

Assumptions:

- Integer size: 4B
- name field's offset and length each occupy 4B
- Null bitmap is omitted

解：记录内存布局（定长部分 + 变长部分）：

- id (INTEGER, NOT NULL, 4B): 偏移 0, 值 23
- name (VARCHAR, 4B + 4B = 8B): 头部的 offset 指向变长区的实际字符串起始位置; length 存储字符串长度 ("Chicago" = 7 字节)
- type (INTEGER, 4B): 偏移 4 + 8 = 12, 值 10
- 变长区: 字符串 "Chicago" (7 字节), 其偏移位置在定长区指定

答案: 记录格式为:

id (4B): 23	name_offset (4B)	name_len (4B): 7	type (4B): 10	"Chicago" (7B)
-------------	------------------	------------------	---------------	----------------

(变长字段在前, 定长字段在后或反之均可, 取决于具体实现。)

3. If NULL values are allowed, the minimum size of a record is: _____ B

解: 若允许 NULL, 定长部分仍占用固定字节数 (id=4 + type=4 = 8B), VARCHAR 字段在 NULL 时 offset 和 length 仍占用空间 (各 4B, 共 8B), 变长区不分配空间。最小记录 = 4 + 4 + 4 + 4 = 16 B (或 8B 若 offset/length 可简化为标志位)。

答案: 按题设假设 offset + length 各 4B, 最小为 **16 B**。

4. Using a slotted-page structure, where: Free space pointer occupies 4B; Record count field occupies 4B; Length and pointer for each record occupy 2B each. The maximum number of records that can be stored in a 4KB page is: _____

解: 页大小 4KB = 4096B。

页头开销:

- Free space pointer: 4B
- Record count: 4B
- 每个 slot entry: length (2B) + pointer (2B) = 4B

设最多 n 条记录, 则:

$$4 + 4 + 4n + \sum (\text{record data}) \leq 4096$$

理论上限 (记录体积极小趋于 0):

$$8 + 4n \leq 4096 \Rightarrow 4n \leq 4088 \Rightarrow n \leq 1022$$

若每条记录至少 1B 有效数据, 则 $8 + 5n \leq 4096$, $n \leq 817$ 。题干未给出记录最小长度, 故按**记录体积趋零**的理论极值作答。

答案: 理论上限为 **1022** (仅计入 slot 开销)。实际操作中受最小记录长度约束。

Problem 5: Indexing

[原卷此处数据缺失] B⁺ 树图缺失, n 值未标。

1. After inserting 27, the root node contains:

A. 6, 17 B. 6 C. 11 D. 11, 17 E. None of the above

解：B⁺ 树插入：从根下到叶插入，若叶满则分裂，分裂可能向上传播至根。27 是否引起根分裂取决于原树结构和 n 值。

答案：需根据初始 B⁺ 树图插入后判断。

2. After insertion, if we then delete 11, the root node contains:

A. 6 B. 17 C. 6 or 17 D. None of the above

解：B⁺ 树删除：找到 11，删除，若叶下溢 (key 数 $< \lceil (n-1)/2 \rceil$) 则尝试 borrow 或 merge，可能向上传播至根。

答案：需根据操作后树状态判断。

3. The table T has a B⁺ Tree index on id and there are 300,000 records in total. The maximum height of the B⁺ Tree is: _____

解：B⁺ 树最大高度出现在节点填充率最低时 (每个非叶节点仅 $\lceil n/2 \rceil$ 个指针，每个叶节点仅 $\lceil (n-1)/2 \rceil$ 个键)。

答案：需已知 n 才能计算： $h_{\max} = \lceil \log_{\lceil n/2 \rceil} (300000 / \lceil (n-1)/2 \rceil) \rceil + 1$ 。

4. The maximum data transfer required to search for a specific record by id is: _____

解：搜索一条记录最多需要从根遍历到叶，每个树层读 1 个节点 (1 transfer)，共 h 层 (根计 1 transfer)。若索引非聚簇，还需 1 transfer 读数据块。

答案：最大 transfer = 树高 h (聚簇) 或 $h + 1$ (非聚簇)。

Problem 6: Query Processing

[原卷回忆细节有误]

1. True or False:

(a) Linear search is applicable in all cases.

(b) Index search is always faster than linear search.

(c) In external sorting, relation r has 400 records, relation s has 3000 records, $B = \dots$ [原卷此处数据缺失]

解：

(a) **True**。线性搜索不要求数据有序或有索引，任何表上都能做。

(b) **False**。当查询选择性很高 (返回大量记录) 时，索引查找需要大量随机 I/O，可能远慢于一次顺序扫描全表。

(c) [原卷此处数据缺失] 缺 B 值，无法判断。

答案: (a) $\checkmark$, (b) $\times$, (c) 待定。

2. The initial number of generate runs for r and s are _____ and _____ respectively.

解: 初始归并段数 $= \lceil b_r/M \rceil$, 其中 b_r 为关系块数, M 为可用内存块数。

答案: 需已知 M 。公式: $\lceil \#blocks/M \rceil$ 。

3. **True or False:** Both r and s require exactly 2 passes during external sorting.

解: 外排序的 pass 数 $= 1 + \lceil \log_{M-1}(b/M) \rceil$ 。关系块数不同, pass 数可能不同。400 条和 3000 条记录若块数差异大, 几乎不可能同时恰好需要 2 pass。

答案: $\times$ (False)

4. After external sorting, when using merge join. The data transfer cost is _____, The seek cost is _____.

解: Merge join 的 transfer cost $= b_r + b_s$ (两个关系各读一次)。Seek cost 取决于缓冲区大小。

答案: Transfer: $b_r + b_s$ 。Seek: 取决于 M 。

5. **True or False:**

(a) If $M > 3$, $M - 2$ blocks should be allocated for the outer relation.

(b) If $M = 20$, then allocating 1 block to r , 18 blocks to s , and 1 block to output would minimize the join cost.

解:

(a) **False**。在 block nested-loop join 中, 若外层用 $M - 2$ 块读入, 内层剩 1 块, 剩余 1 块用于 output。这样外层一次可读入 $M - 2$ 块块减少内层扫描次数, 确实是最优策略, 而非“ $M > 3$ ”就自动成立——还需 $M \geq 3$ 且两块给 output。措辞“should”方向对但“ $M > 3$ ”条件粗糙。

(b) **True** (在特定假设下)。将较小的 r 用 1 块读入作为外层, 较大的 s 用 $M - 2 = 18$ 块作为内层一次性读入, 每轮外扫描内层一次, 总开销最小。

答案: (a) 倾向于 $\times$ (条件不严谨), (b) $\checkmark$ (当 $b_s \gg b_r$ 时)。

Problem 7: Query Optimization

[原卷回忆细节有误]

1. The following equivalence relations are incorrect:

A. $\sigma_{p \wedge q}(r) = \sigma_p(\sigma_q(r))$

B. $\sigma_p(r \bowtie s) = \sigma_p(r) \bowtie s$ (当 p 仅含 r 的属性)

C. $\pi_A(r \bowtie s) = \pi_A(r) \bowtie \pi_A(s)$

D. $\sigma_p(r \cup s) = \sigma_p(r) \cup \sigma_p(s)$

解:

- A. 正确。合取可级联选择。
- B. 正确。选择下推。
- C. **不正确**。若 join 涉及 r 和 s 的不同属性, 先投影各自到 A 再 join 可能丢失 join 条件所需的属性, 导致无法等值连接。投影下推需保留下推所需的列。
- D. 正确。选择对并集分配律。

答案: C

```
CREATE TABLE student (  
    sid    INTEGER PRIMARY KEY,  
    name   VARCHAR(20) NOT NULL,  
    gender CHAR(1) CHECK (gender IN ('M', 'F'))  
);
```

```
CREATE TABLE grade (  
    sid    INTEGER PRIMARY KEY,  
    grade  INTEGER,  
    FOREIGN KEY (sid) REFERENCES student(sid)  
);
```

```
SELECT s.sid  
FROM student s, grade g  
WHERE s.sid = g.sid AND grade > 107;
```

Assumptions:

- `sid` has 50 distinct values
- `grade` ranges from 60 to 120 (inclusive), uniformly distributed

2. The selectivity of condition `s.sid = g.sid` is: _____

解: `sid` 有 50 个 distinct 值。等值连接 $s.sid = g.sid$ 的 $\text{selectivity} = 1 / \max(V(s.sid, student), V(sid, grade)) = 1 / \max(50, 50) = 1/50$ 。

含义: 在笛卡尔积中, 仅 $1/50$ 的行连接条件成立。

答案: $1/50 = 0.02$

3. The selectivity of condition `grade > 107` is: _____

解: `grade` 在 $[60, 120]$ 上均匀分布, 共 $120 - 60 + 1 = 61$ 种取值。`grade > 107` 即 `grade` $\in [108, 120]$, 共 $120 - 108 + 1 = 13$ 种取值。均匀分布下 $\text{selectivity} = 13/61 \approx 0.213$ 。

答案: $13/61 \approx 0.213$

Problem 8: Transaction and Concurrency Control

1. True or False:

- (a) A conflict-serializable schedule must be a recoverable schedule.
- (b) Conflict serializability corresponds to a unique serial schedule.
- (c) 2PL (Two-Phase Locking) ensures recoverability.
- (d) Strict 2PL ensures recoverability.

解:

- (a) **False**。冲突可串行化 $\nRightarrow$ 可恢复。反例: T_2 读了 T_1 写的脏数据并 commit, 然后 T_1 abort。这是冲突可串行化但不恢复 (脏读)。
- (b) **False**。冲突可串行化说明存在某个串行调度的执行效果与之等价, 但该串行调度未必唯一。含无关事务并行时可能有多个等价的串行顺序。
- (c) **False**。基本 2PL 不保证可恢复性——事务可在 commit 前读未提交数据, 可能脏读。
- (d) **True**。Strict 2PL 要求事务直到 commit 才释放排他锁, 杜绝了读未提交数据的可能, 因此保证可恢复性 (且保证无级联回滚)。

答案: (a) $\times$, (b) $\times$, (c) $\times$, (d) $\checkmark$

2. Draw the precedence graph. What are the edges?

T1	T2	T3
Read(A)		
Write(A)		
Write(A)		
	Read(B)	
	Read(B)	
	Write(B)	
		Read(B)
		Read(C)
		Write(C)

Options: A, B, C, D. None of above. 原题 B 选项有问题

解: 按行序提取冲突:

- T_1 Read(A) $\rightarrow$ 无前驱 (A 首次访问)
- T_1 Write(A) $\rightarrow$ 与前面的自身 Read 不构成跨事务冲突
- T_2 Write(A) $\rightarrow$ 冲突于 T_1 的 Write(A): $T_1 \rightarrow T_2$ 也冲突于 T_1 的 Read(A) 吗? 读在写之前, $T_1 : R(A) \rightarrow T_2 : W(A)$, 这是 $T_1 \rightarrow T_2$ 。
- T_2 Read(B) $\rightarrow$ B 首次被访问, 无冲突
- T_3 Read(B) $\rightarrow$ 与 T_2 的 Read(B) 不冲突 (两个读)
- T_3 Write(B) $\rightarrow$ 冲突于 T_2 Read(B): $T_2 \rightarrow T_3$

- T_3 Read(C) $\rightarrow$ C 首次
- T_3 Write(C) $\rightarrow$ 与自身不冲突

边集合: $T_1 \rightarrow T_2$, $T_2 \rightarrow T_3$ 。无环。

答案: 边为 $T_1 \rightarrow T_2$ 、 $T_2 \rightarrow T_3$ 。

3. Is this schedule conflict-serializable?

解: 前驱图无环 $\Rightarrow$ 冲突可串行化。串行等价顺序: $T_1 \rightarrow T_2 \rightarrow T_3$ 。

答案: Yes (是)

4. True or False: Deadlock

- (a) Deadlock can be represented in a precedence graph.
- (b) The Tree Protocol can prevent deadlocks.
- (c) Rigorous 2PL can prevent deadlocks.
- (d) The Wait-Die scheme can prevent deadlocks.

解:

- (a) **False**。Precedence graph 用于判断冲突可串行化，刻画事务间的操作冲突。死锁的表示需 **wait-for graph** (等待图)，两者不是同一个图。死锁意味着 cycles in wait-for graph，与 precedence graph 无关。
- (b) **True**。树协议通过限制锁的获取顺序（沿树从根到叶，先锁父再锁子），避免了循环等待，从而预防死锁。
- (c) **False**。Rigorous 2PL (严格两阶段锁) 保证可串行化和级联回滚预防，但**不能预防死锁**——事务仍可因锁升级冲突而互相等待形成循环。
- (d) **True**。Wait-Die 基于时间戳的抢占式死锁预防方案，旧事务等新事务 (wait)，新事务不等旧事务 (die 后重试)，保证无法形成等待环。

答案: (a) $\times$, (b) $\checkmark$, (c) $\times$, (d) $\checkmark$

Problem 9: Recovery

3000: <T1, 6421.1, 10, 20>
3001: <T2, start>
3002: <T3, start>
3003: <T2, 3462.1, 30, 40>
3004: <T3, 7923.1, 50, 60>
3005: <T2, 3462.1, 40, 70>
3006: <T1, 6421.1, 20, 80>
3007: <T3, commit>
3008: checkpoint

DirtyPageTable

Page	PageLSN	RecLSN
3462	3005	3005
6421	3006	3000
7923	3004	3004

TXN LastLSN

T1	3006
T2	3005

3009: <T4, start>
3010: <T4, 5235.1, 60, 90>
3011: <T1, 6421.1, 80, 100>
3012: <T1, commit>

1. The Analysis phase starts at:

A. 3000 B. 3001 C. 3002 D. 3008 E. None of the above

解：Analysis 阶段从**最近的完整 checkpoint** 开始。最近 checkpoint 为 LSN 3008。从 checkpoint 处开始向前扫描（forward scan）至日志末尾，重建 checkpoint 之后发生的 dirty page table 和活跃事务表。

答案：D. 3008

2. After the Analysis phase completes, what are the DirtyPageTable and undo-list? Fill in the tables below (leave a row empty if an entry does not exist).

Page	PageLSN	RecLSN	TXN	LastLSN
3462	—	—	T1	—
5235	—	—	T2	—
6421	—	—	T3	—
7923	—	—	T4	—

解：Analysis 阶段：从 LSN 3008（checkpoint）开始，继承其脏页表和活跃事务表，再正向扫描 3009–3012。

初始状态（checkpoint 时）：

- DirtyPageTable: P3462(3005,3005), P6421(3006,3000), P7923(3004,3004)
- TXN table: T1(LastLSN=3006), T2(LastLSN=3005), T3 已 commit (不在活跃表中)

扫描 3009–3012:

- 3009: T4 start → 加入 TXN table, LastLSN=3009
- 3010: T4 upd P5235 → 5265 入 DirtyPageTable(5235, 3010, 3010); T4 的 LastLSN 更新为 3010
- 3011: T1 upd P6421 → 更新 DirtyPageTable(6421, 3011, 3000) (PageLSN 更新为 3011, RecLSN 保持最早的 3000); T1 LastLSN 更新为 3011
- 3012: T1 commit → T1 移出 TXN table

Analysis 结束:

- Winner (已 commit): T1 (分析前已 commit, 扫描中又 commit), T3
- Loser (undo list): T2 (未 commit), T4 (未 commit)

答案:

Winner: T1, T3。

3. The Redo phase starts at:

- A. 3000 B. 3001 C. 3002 D. 3008 E. None of the above

解: Redo 从脏页表中最小的 RecLSN 开始。查上表: P3462: RecLSN=3005, P5235: RecLSN=3010, P6421: RecLSN=3000, P7923: RecLSN=3004。最小 RecLSN = **3000**。

答案: A. 3000

4. During the Redo phase, which log entries are skipped?

- A. 3000 B. 3003 C. 3010 D. 3011 E. None of the above

解: Redo 阶段从 LSN 3000 开始重新执行**所有** update 日志 (包括 loser 的修改), 但跳过以下情况:

- 页不在脏页表中
- 页的 PageLSN $\geq$ 当前 LSN (页已更新过)

逐个检查 3000–3012:

- 3000 (T1, P6421): 页在 DPT, $\text{RecLSN} \leq 3000$, $\text{PageLSN}=3006 \geq 3000$? 实际执行 Redo 时按 LSN 递增遍历日志。3000 到达时 P6421 的 PageLSN 尚为初始值 (<3000), 不跳过。执行 3000。
- 3003 (T2, P3462): 不跳过, 执行。
- 3010 (T4, P5235): 不跳过, 执行。
- 3011 (T1, P6421): 不跳过, 执行。

所有 update log 均不跳过。

答案: E. None of the above

5. The first log entry to be undone is:

- A. 3011 B. 3010 C. 3006 D. 3005 E. None of the above

解：Undo 按 LastLSN 从大到小反向遍历 loser 事务的 prevLSN 链。Loser 为 T2 (LastLSN=3005) 和 T4 (LastLSN=3010)。

按 LastLSN 排序：T4(3010) > T2(3005)，先 undo T4 的 3010。

答案：B. 3010

6. The LSN of the CLR (Compensation Log Record) for log entry 3005 is:

A. 3013 B. 3014 C. 3015 D. 3016 E. None of the above

解：Undo 阶段按 LastLSN 降序处理 loser。先 undo T4(LSN 3010) → 产生 CLR@3013。再 undo T2(LSN 3005) → 产生 CLR@3014。再沿 T2 的 prevLSN 链往上找：3005 是 T2 的最后一条，前一操作是 3003。

CLR@3013: undo LSN 3010; CLR@3014: undo LSN 3005。所以 LSN 3005 对应的 CLR 的 LSN 是 3014。

答案：B. 3014

7. After recovery completes, what are the values of 3462.1 and 6421.1?

A. 70, 10 B. 30, 10 C. 30, 100 D. 70, 100 E. None of the above

解：追溯最终值：

P3462.1：日志序列为 T2 写入 3003(30→40)、3005(40→70)。Redo 阶段从 RecLSN 起正向重做：3003 和 3005 均重做 → 当前 =70。但 T2 是 loser! Undo 阶段沿 prevLSN 链反向撤销：undo 3005 (恢复为 40) → undo 3003 (恢复为 30)。最终 3462.1 = **30**。

P6421.1：日志序列为 T1 写入 3000(10→20)、3006(20→80)、3011(80→100)。Redo 重做 3000、3006、3011 → 当前 =100。T1 是 winner (已 commit)，不 undo。最终 6421.1 = **100**。

答案：C. 30, 100